SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 14-15

Árboles y Recursividad

Utiliza recursividad en la programación

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	14 - 15
Tarea de aprendizaje	Utiliza recursividad en la programación
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Verificar o instalar el JDK 4
1.2 Instalar la extensión de Java en VS Code 4
1.3 Opción A: abrir el proyecto entregado 4
1.4 Opción B: crear el proyecto desde cero 4
1.5 Ejecutar los programas 5
Parte 2. Conceptos clave 6
2.1 Recursividad 6
2.2 Árboles: terminología 6
2.3 Árbol binario y árbol binario de búsqueda 7
2.4 Recorridos de un árbol (traversal) 8
2.5 El juego Sí/No 8
Parte 3. Prácticas en Visual Studio Code 10
Práctica 01. Funciones recursivas (Semana 14) 10
Práctica 02. Recursivo vs. iterativo y problemas clásicos (Semana 14) 12
Práctica 03. Definir y crear un árbol binario (Semana 14) 13
Práctica 04. Recorridos pre-order, in-order y post-order (Semana 15) 16
Práctica 05. Aplicación: árbol de expresión (Semana 15) 18
Práctica 06. Proyecto: Juego Sí/No que aprende (Semana 15) 20
Parte 4. Errores frecuentes 25
Parte 5. Ejercicios propuestos 25
Preguntas de reflexión 25
Lista de cotejo 26

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía (dos semanas) serás capaz de:

- **Usar** la recursividad para resolver problemas, identificando el caso base y el caso recursivo.
- **Definir y crear** la estructura de datos **árbol binario** en Java.
- **Usar** los algoritmos de recorrido **pre-order** y **post-order** (además de in-order).
- **Construir** el juego Sí/No, un árbol que aprende y que es la base de los árboles de decisión de la IA.

Operación	Dónde se trabaja en esta guía
Usa la recursividad	Parte 2 (sección 2.1), Prácticas 01 y 02
Define y crea la estructura Árbol	Parte 2 (secciones 2.2 y 2.3), Práctica 03
Usa algoritmos pre-order traversal	Parte 2 (sección 2.4), Prácticas 04, 05 y 06
Usa algoritmos post-order traversal	Parte 2 (sección 2.4), Prácticas 04, 05 y 06
Juego Sí/No	Parte 2 (sección 2.5), Práctica 06

Semana	Contenido sugerido
14	Partes 1 y 2 (secciones 2.1 a 2.3). Prácticas 01, 02 y 03. Ejercicios 1 y 2.
15	Parte 2 (secciones 2.4 y 2.5). Prácticas 04, 05 y 06. Ejercicios 3 a 5.

Materiales

- Computadora con Windows 10/11, **JDK 17 o superior** y Visual Studio Code con **Extension Pack for Java**.
- Proyecto **Semana14_15_Arboles_Java.zip** entregado por el instructor.

Parte 1. Preparación del entorno en Visual Studio Code

Esta guía trae el proyecto de VS Code **Semana14_15_Arboles_Java** con todas las clases listas. Puedes abrirlo directamente (opción A) o crearlo tú desde cero y escribir cada clase (opción B, recomendada para practicar).

1.1 Verificar o instalar el JDK

Abre una terminal (PowerShell) y escribe:

Terminal de VS Code

```
java -version
javac -version
```

▶ Resultado esperado (versión 17 o superior)

```
java version "21.0.x" ...
javac 21.0.x
```

Si aparece **«java no se reconoce como un comando»**, instala el **JDK 21** (Oracle JDK desde oracle.com/java o Eclipse Temurin desde adoptium.net). Durante la instalación marca las opciones **Add to PATH** y **Set JAVA_HOME**, y luego cierra y vuelve a abrir VS Code.

> ⚠ **Atención:** Los programas de esta guía usan `record`, `switch` con flechas y otras novedades de Java moderno. Necesitas **Java 17 o superior**; con Java 8 o 11 aparecerán errores de compilación.

1.2 Instalar la extensión de Java en VS Code

1. En VS Code abre la vista de **Extensiones** (Ctrl + Shift + X).
2. Busca **Extension Pack for Java** (de Microsoft) e instálala. Incluye el soporte del lenguaje, el depurador, el ejecutor de pruebas y el administrador de proyectos.
3. Reinicia VS Code si te lo solicita.

1.3 Opción A: abrir el proyecto entregado

1. Descomprime **Semana14_15_Arboles_Java.zip** en tu carpeta **Documentos**.
2. En VS Code entra a **Archivo > Abrir carpeta…** y selecciona la carpeta **Semana14_15_Arboles_Java** (la que contiene `src`, no la carpeta de arriba).
3. Elige **Sí, confío en los autores** si VS Code lo pregunta.
4. Espera a que en la barra inferior aparezca **Java: Ready** (la primera vez puede tardar un minuto).

1.4 Opción B: crear el proyecto desde cero

1. Pulsa **Ctrl + Shift + P**, escribe **Java: Create Java Project** y elige **No build tools**.
2. Selecciona **Documentos** como ubicación y escribe el nombre **Semana14_15_Arboles_Java**.
3. VS Code crea las carpetas `src`, `lib`, `bin` y el archivo `src/App.java` (puedes borrarlo).

4. Crea una carpeta **datos** junto a `src`. En la carpeta `src` crea cada clase con clic derecho > **Nuevo archivo**, usando el nombre exacto que indica la guía.

La estructura final del proyecto debe quedar así:

Estructura del proyecto

```
Semana14_15_Arboles_Java/
├── .vscode/settings.json
├── datos/
│   └── animales.txt           (árbol inicial del juego Sí/No)
└── src/
    ├── Nodo.java              (nodo genérico de árbol binario)
    ├── ArbolBinario.java      (árbol binario de búsqueda)
    ├── P01_Recursividad.java
    ├── P02_RecursivoVsIterativo.java
    ├── P03_ArbolBinario.java
    ├── P04_Recorridos.java
    ├── P05_ArbolExpresion.java
    └── P06_JuegoSiNo.java
```

⚠ **Atención:** En Java el **nombre del archivo** debe ser idéntico al de la clase pública, respetando mayúsculas: la clase `public class P01_DatosInformacion` va en `P01_DatosInformacion.java`.

1.5 Ejecutar los programas

1. Abre el archivo (por ejemplo `P01_Recursividad.java`).
2. Haz clic en **Run** que aparece encima de `public static void main`, o en el botón ▶ de arriba a la derecha.
3. El resultado aparece en la **TERMINAL**. Si el programa pide datos con `Scanner`, escríbelos allí y presiona Enter.
4. Compara lo que ves con el recuadro verde **Resultado esperado**.

También puedes compilar y ejecutar desde la terminal, ubicado en la carpeta del proyecto:

Terminal de VS Code

```
javac -encoding UTF-8 -d bin src\*.java
java -cp bin P01_Recursividad
```

💡 **Consejo:** Si las tildes o la ñ se ven como símbolos raros (por ejemplo `INFORMACI?N`), escribe una vez `chcp 65001` en la terminal y vuelve a ejecutar.

💡 **Consejo:** Para depurar: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y elige **Debug** encima del `main`. Con **F10** avanzas línea por línea y con **F11** entras a un método; el panel **Variables** muestra los valores en cada momento.

Parte 2. Conceptos clave

2.1 Recursividad

Un método es **recursivo** cuando **se llama a sí mismo** para resolver una versión más pequeña del mismo problema. Es como las muñecas rusas (matrioskas): para llegar a la más pequeña abres una muñeca, y dentro encuentras otra igual pero más chica.

Parte	Función	En factorial(n)
Caso base	Condición que **detiene** la recursión y devuelve un resultado directo.	`if (n <= 1) return 1;`
Caso recursivo	Llama al mismo método con un problema **más pequeño**, acercándose al caso base.	`return n * factorial(n - 1);`

Cada llamada se guarda en la **pila de llamadas** (call stack) hasta que llega al caso base; luego las llamadas se resuelven en orden inverso:

Pila de llamadas de factorial(4)

```
factorial(4) = 4 * factorial(3)          ↓ se apilan las llamadas
               factorial(3) = 3 * factorial(2)
                              factorial(2) = 2 * factorial(1)
                                             factorial(1) = 1   ← CASO BASE
                              factorial(2) = 2 * 1 = 2       ↑ se resuelven
               factorial(3) = 3 * 2 = 6
factorial(4) = 4 * 6 = 24
```

	Recursivo	Iterativo (bucle)
Cómo repite	El método se llama a sí mismo.	Usa `for` o `while`.
Ventaja	Código corto y natural para **árboles** y problemas que se dividen.	Más rápido y usa menos memoria.
Riesgo	`StackOverflowError` si no hay caso base o la recursión es muy profunda.	Bucle infinito si la condición nunca cambia.

> ⚠ **Atención:** Toda función recursiva **debe tener un caso base** y cada llamada debe **acercarse** a él. Si no, el programa se llama sin fin hasta llenar la pila y termina con **StackOverflowError** (Práctica 02).

2.2 Árboles: terminología

Un **árbol** es una estructura de datos **jerárquica** formada por **nodos** conectados por **aristas**. A diferencia de un arreglo o una lista, que son lineales, el árbol se ramifica. Es una estructura **recursiva**: cada hijo es, a su vez, la raíz de un subárbol.

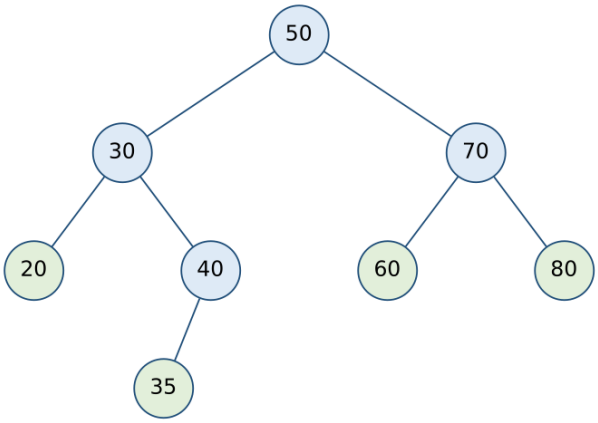


Figura 1. Árbol binario de búsqueda de la Práctica 03 (hojas en verde).

Término	Significado	En la Figura 1
Raíz	Nodo superior; no tiene padre.	50
Padre / hijo	Un nodo es padre de los nodos que cuelgan directamente de él.	30 es padre de 20 y 40
Hoja	Nodo sin hijos.	20, 35, 60, 80
Nodo interno	Nodo que tiene al menos un hijo.	50, 30, 70, 40
Subárbol	Un nodo con todos sus descendientes.	40 → 35
Nivel	Distancia a la raíz (la raíz está en el nivel 1).	35 está en el nivel 4
Altura	Número de niveles del camino más largo.	4

2.3 Árbol binario y árbol binario de búsqueda

En un **árbol binario** cada nodo tiene **como máximo dos hijos**: izquierdo y derecho. En Java se representa con una clase Nodo que guarda un dato y dos referencias:

Archivo: src/Nodo.java

```
// Nodo.java - Un nodo de árbol binario: un dato y dos enlaces (hijos)
public class Nodo<T> {
  T dato;
  Nodo<T> izquierdo;     // hijo izquierdo
  Nodo<T> derecho;       // hijo derecho

  public Nodo(T dato) {
    this.dato = dato;
  }

  public Nodo(T dato, Nodo<T> izquierdo, Nodo<T> derecho) {
    this.dato = dato;
    this.izquierdo = izquierdo;
    this.derecho = derecho;
  }

  public boolean esHoja() {
    return izquierdo == null && derecho == null;
  }
}
```

El `<T>` hace que el nodo sea **genérico**: puede guardar números (`Nodo<Integer>`) o textos (`Nodo<String>`). Un **árbol binario de búsqueda (ABB)** agrega una regla: los valores **menores** van a la izquierda y los **mayores** a la derecha. Así, buscar un dato es muy rápido porque en cada paso se descarta la mitad del árbol.

2.4 Recorridos de un árbol (traversal)

Recorrer un árbol es visitar todos sus nodos una sola vez. Los tres recorridos clásicos son recursivos y solo se diferencian en **cuándo se visita la raíz**:

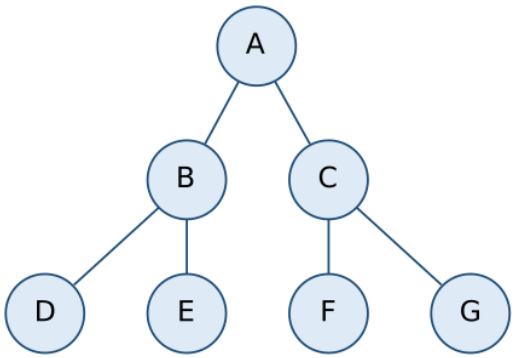


Figura 2. Árbol de la Práctica 04.

Recorrido	Orden	Resultado (Figura 2)	Uso típico
Pre-order	**Raíz** → izquierda → derecha	A B D E C F G	Copiar o **guardar** un árbol en un archivo; notación prefija.
In-order	Izquierda → **raíz** → derecha	D B E A F C G	Obtener los datos de un ABB **ordenados**.
Post-order	Izquierda → derecha → **raíz**	D E B F G C A	**Evaluar** expresiones, contar o liberar nodos (primero los hijos).

Plantilla de los recorridos

```
void preOrden(Nodo n)  { if (n == null) return;
                         VISITAR(n); preOrden(n.izq); preOrden(n.der); }

void inOrden(Nodo n)   { if (n == null) return;
                         inOrden(n.izq); VISITAR(n); inOrden(n.der); }

void postOrden(Nodo n) { if (n == null) return;
                         postOrden(n.izq); postOrden(n.der); VISITAR(n); }
```

> 💡 **Consejo:** Truco para recordar: el nombre indica la posición de la **raíz**: **pre** = antes, **in** = en medio, **post** = después de los hijos.

2.5 El juego Sí/No

El juego Sí/No (también llamado «20 preguntas» o «Adivina el animal») usa un árbol binario donde los **nodos internos son preguntas** y las **hojas son respuestas**. Cada respuesta «sí» lleva a la rama izquierda

y cada «no» a la derecha. Cuando el programa se equivoca, **aprende**: convierte la hoja en una nueva pregunta con dos hijos.

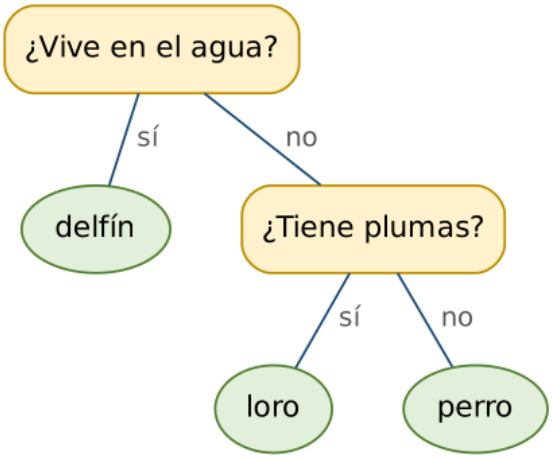


Figura 3. Árbol inicial del juego (datos/animales.txt).

Relación con la IA: este juego es un **sistema experto** muy simple y un **árbol de decisión**: cada pregunta divide las posibilidades en dos. En la Semana 16, el algoritmo **ID3** construirá árboles de decisión automáticamente, eligiendo él mismo las mejores preguntas a partir de los datos.

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Funciones recursivas (Semana 14)

Programarás cuatro funciones recursivas clásicas. La función factorial imprime una **traza** con sangría para que veas cómo se apilan y resuelven las llamadas.

Archivo: src/P01_Recursividad.java

```
// P01_Recursividad.java - Funciones recursivas: caso base + caso recursivo
public class P01_Recursividad {

  // n! = n * (n-1)!     con     0! = 1
  static long factorial(int n, int nivel) {
    String sangria = "  ".repeat(nivel);
    System.out.println(sangria + "factorial(" + n + ") llamado");
    if (n <= 1) {                                        // CASO BASE: se detiene
      System.out.println(sangria + "factorial(" + n + ") = 1   <- caso base");
      return 1;
    }
    long resultado = n * factorial(n - 1, nivel + 1);     // CASO RECURSIVO
    System.out.println(sangria + "factorial(" + n + ") = " + resultado);
    return resultado;
  }

  // Suma de los dígitos: 4725 -> 4 + 7 + 2 + 5 = 18
  static int sumaDigitos(int n) {
    if (n < 10) return n;                     // caso base: un solo dígito
    return n % 10 + sumaDigitos(n / 10);      // último dígito + resto del número
  }

  // base^exp usando recursividad
  static long potencia(long base, int exp) {
    if (exp == 0) return 1;
    return base * potencia(base, exp - 1);
  }

  static int llamadas = 0;
  // Fibonacci: f(n) = f(n-1) + f(n-2)   con   f(0) = 0, f(1) = 1
  static long fibonacci(int n) {
    llamadas++;
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
  }

  public static void main(String[] args) {
    System.out.println("=== Traza de factorial(4) ===");
    System.out.println("Resultado final: " + factorial(4, 0));

    System.out.println("\nsumaDigitos(4725) = " + sumaDigitos(4725));
    System.out.println("potencia(2, 10)   = " + potencia(2, 10));

    System.out.println("\nSerie de Fibonacci:");
    for (int i = 0; i <= 10; i++) System.out.print(fibonacci(i) + " ");
    llamadas = 0;
    System.out.println("\nfibonacci(25) = " + fibonacci(25) + " necesitó " + llamadas
        + " llamadas");
  }
}
```

▶ **Resultado esperado en la terminal**

```
=== Traza de factorial(4) ===
factorial(4) llamado
  factorial(3) llamado
    factorial(2) llamado
      factorial(1) llamado
      factorial(1) = 1   <- caso base
    factorial(2) = 2
  factorial(3) = 6
factorial(4) = 24
Resultado final: 24

sumaDigitos(4725) = 18
potencia(2, 10)   = 1024

Serie de Fibonacci:
0 1 1 2 3 5 8 13 21 34 55
fibonacci(25) = 75025 necesitó 242785 llamadas
```

Análisis: la traza de `factorial(4)` muestra la pila de la sección 2.1: las llamadas bajan hasta el caso base y luego suben multiplicando. Observa también que `fibonacci(25)` hizo **242 785 llamadas**, porque calcula los mismos valores muchas veces: la recursión es elegante, pero no siempre eficiente.

> 💡 **Consejo:** Pon un punto de interrupción en la línea `if (n <= 1)` de `factorial` y ejecuta con **Debug**. En el panel **CALL STACK** verás las llamadas apiladas una sobre otra.

Práctica 02. Recursivo vs. iterativo y problemas clásicos (Semana 14)

Archivo: src/P02_RecursivoVsIterativo.java

```
// P02_RecursivoVsIterativo.java - Comparación y problemas clásicos de recursividad
public class P02_RecursivoVsIterativo {

  static long factorialIterativo(int n) {
    long r = 1;
    for (int i = 2; i <= n; i++) r *= i;          // usa un bucle
    return r;
  }

  static long factorialRecursivo(int n) {
    return (n <= 1) ? 1 : n * factorialRecursivo(n - 1);   // se llama a sí mismo
  }

  // Un texto es palíndromo si su primera y última letra son iguales
  // y el texto del medio también es palíndromo
  static boolean esPalindromo(String t) {
    if (t.length() <= 1) return true;                      // caso base
    if (t.charAt(0) != t.charAt(t.length() - 1)) return false;
    return esPalindromo(t.substring(1, t.length() - 1));    // reduce el problema
  }

  // Búsqueda binaria recursiva en un arreglo ORDENADO
  static int buscar(int[] a, int clave, int ini, int fin) {
    if (ini > fin) return -1;                              // caso base: no está
    int medio = (ini + fin) / 2;
    System.out.println("   revisando posiciones " + ini + " a " + fin + " -> medio = "
        + a[medio]);
    if (a[medio] == clave) return medio;                   // caso base: encontrado
    if (clave < a[medio]) return buscar(a, clave, ini, medio - 1);
    return buscar(a, clave, medio + 1, fin);
  }

  // ERROR TÍPICO: recursión SIN caso base
  static int sinCasoBase(int n) {
    return sinCasoBase(n + 1);
  }

  public static void main(String[] args) {
    System.out.println("factorial(10) iterativo = " + factorialIterativo(10));
    System.out.println("factorial(10) recursivo = " + factorialRecursivo(10));

    for (String p : new String[]{"reconocer", "anilina", "senati"}) {
      System.out.println("¿\"" + p + "\" es palíndromo? " + esPalindromo(p));
    }

    int[] notas = {5, 8, 10, 11, 13, 14, 16, 17, 19, 20};
    System.out.println("\nBuscando 16 en el arreglo ordenado:");
    System.out.println("   encontrado en la posición " + buscar(notas, 16, 0,
        notas.length - 1));

    try {
      sinCasoBase(1);
    } catch (StackOverflowError e) {
      System.out.println("\nsinCasoBase() provocó StackOverflowError: "
          + "la pila de llamadas se llenó.");
    }
  }
}
```

▶ **Resultado esperado en la terminal**

```
factorial(10) iterativo = 3628800
factorial(10) recursivo = 3628800
¿"reconocer" es palíndromo? true
¿"anilina" es palíndromo? true
¿"senati" es palíndromo? false

Buscando 16 en el arreglo ordenado:
   revisando posiciones 0 a 9 -> medio = 13
   revisando posiciones 5 a 9 -> medio = 17
   revisando posiciones 5 a 6 -> medio = 14
   revisando posiciones 6 a 6 -> medio = 16
   encontrado en la posición 6

sinCasoBase() provocó StackOverflowError: la pila de llamadas se llenó.
```

Análisis: el palíndromo y la búsqueda binaria **reducen el problema** en cada llamada: quitan las letras de los extremos o descartan la mitad del arreglo. La búsqueda encontró el 16 revisando solo 4 elementos de 10. El último caso muestra qué pasa sin caso base: `StackOverflowError`.

Práctica 03. Definir y crear un árbol binario (Semana 14)

Primero crea `Nodo.java` (sección 2.3) y luego la clase `ArbolBinario`. **Todos sus métodos son recursivos**: cada uno resuelve el problema para la raíz y delega el resto a los subárboles izquierdo y derecho.

Archivo: src/ArbolBinario.java

```
// ArbolBinario.java - Árbol binario de búsqueda (ABB) con métodos recursivos
import java.util.ArrayList;
import java.util.List;

public class ArbolBinario {
  private Nodo<Integer> raiz;

  // INSERTAR: menores a la izquierda, mayores a la derecha
  public void insertar(int valor) {
    raiz = insertar(raiz, valor);
  }
  private Nodo<Integer> insertar(Nodo<Integer> n, int valor) {
    if (n == null) return new Nodo<>(valor);  // caso base: lugar vacío
    if (valor < n.dato) n.izquierdo = insertar(n.izquierdo, valor);
    else if (valor > n.dato) n.derecho = insertar(n.derecho, valor);
    return n;  // (los repetidos se ignoran)
  }

  public boolean buscar(int valor) { return buscar(raiz, valor); }
  private boolean buscar(Nodo<Integer> n, int valor) {
    if (n == null) return false;
    if (valor == n.dato) return true;
    return valor < n.dato ? buscar(n.izquierdo, valor) : buscar(n.derecho, valor);
  }

  public int contarNodos() { return contarNodos(raiz); }
  private int contarNodos(Nodo<Integer> n) {
    if (n == null) return 0;
    return 1 + contarNodos(n.izquierdo) + contarNodos(n.derecho);
  }

  public int contarHojas() { return contarHojas(raiz); }
  private int contarHojas(Nodo<Integer> n) {
    if (n == null) return 0;
    if (n.esHoja()) return 1;
    return contarHojas(n.izquierdo) + contarHojas(n.derecho);
  }

  // Altura = número de niveles del camino más largo desde la raíz
  public int altura() { return altura(raiz); }
  private int altura(Nodo<Integer> n) {
    if (n == null) return 0;
    return 1 + Math.max(altura(n.izquierdo), altura(n.derecho));
  }

  // RECORRIDOS (traversal)
  public List<Integer> preOrden() {
    List<Integer> lista = new ArrayList<>();
    preOrden(raiz, lista);
    return lista;
  }
  public List<Integer> inOrden() {
    List<Integer> lista = new ArrayList<>();
    inOrden(raiz, lista);
    return lista;
  }
  public List<Integer> postOrden() {
    List<Integer> lista = new ArrayList<>();
    postOrden(raiz, lista);
    return lista;
  }

  private void preOrden(Nodo<Integer> n, List<Integer> l) {   // RAÍZ - izq - der
    if (n == null) return;
    l.add(n.dato);
    preOrden(n.izquierdo, l);
    preOrden(n.derecho, l);
  }
```

```
  private void inOrden(Nodo<Integer> n, List<Integer> l) {    // izq - RAÍZ - der
    if (n == null) return;
    inOrden(n.izquierdo, l);
    l.add(n.dato);
    inOrden(n.derecho, l);
  }
  private void postOrden(Nodo<Integer> n, List<Integer> l) {  // izq - der - RAÍZ
    if (n == null) return;
    postOrden(n.izquierdo, l);
    postOrden(n.derecho, l);
    l.add(n.dato);
  }

  // Dibuja el árbol "acostado": la raíz a la izquierda, el hijo derecho arriba
  public void imprimir() { imprimir(raiz, 0); }
  private void imprimir(Nodo<Integer> n, int nivel) {
    if (n == null) return;
    imprimir(n.derecho, nivel + 1);
    System.out.println("      ".repeat(nivel) + "-> " + n.dato);
    imprimir(n.izquierdo, nivel + 1);
  }
}
```

Cada operación pública (`insertar`, `contarNodos`…) llama a una versión **privada** que recibe un nodo. Así quien usa el árbol no necesita conocer los nodos internos. Ahora crea el programa de prueba:

Archivo: src/P03_ArbolBinario.java

```
// P03_ArbolBinario.java - Crear y consultar un árbol binario de búsqueda
public class P03_ArbolBinario {
  public static void main(String[] args) {
    ArbolBinario arbol = new ArbolBinario();
    int[] valores = {50, 30, 70, 20, 40, 60, 80, 35};
    System.out.print("Insertando: ");
    for (int v : valores) {
      System.out.print(v + " ");
      arbol.insertar(v);
    }
    System.out.println("\n\nÁrbol (gíralo 90° a la derecha para verlo de pie):");
    arbol.imprimir();

    System.out.println("\nNodos totales : " + arbol.contarNodos());
    System.out.println("Hojas         : " + arbol.contarHojas());
    System.out.println("Altura        : " + arbol.altura());
    System.out.println("¿Existe el 40?: " + arbol.buscar(40));
    System.out.println("¿Existe el 45?: " + arbol.buscar(45));
    System.out.println("Pre-orden : " + arbol.preOrden());
    System.out.println("In-orden  : " + arbol.inOrden() + "  <- sale ORDENADO");
    System.out.println("Post-orden: " + arbol.postOrden());
  }
}
```

▶ **Resultado esperado en la terminal**

```
Insertando: 50 30 70 20 40 60 80 35

Árbol (gíralo 90° a la derecha para verlo de pie):
            -> 80
      -> 70
            -> 60
-> 50
            -> 40
                  -> 35
      -> 30
            -> 20

Nodos totales : 8
Hojas         : 4
Altura        : 4
¿Existe el 40?: true
¿Existe el 45?: false
Pre-orden : [50, 30, 20, 40, 35, 70, 60, 80]
In-orden  : [20, 30, 35, 40, 50, 60, 70, 80]  <- sale ORDENADO
Post-orden: [20, 35, 40, 30, 60, 80, 70, 50]
```

Análisis: el dibujo «acostado» corresponde a la Figura 1. El recorrido **in-order** de un ABB siempre sale **ordenado** de menor a mayor. `contarNodos` usa la idea recursiva «1 (yo) + los nodos de mi izquierda + los de mi derecha».

Práctica 04. Recorridos pre-order, in-order y post-order (Semana 15)

Construirás a mano el árbol de la Figura 2 y verás, paso a paso, en qué orden visita cada recorrido los nodos. La sangría indica el nivel del nodo.

Archivo: src/P04_Recorridos.java

```
// P04_Recorridos.java - Pre-order, in-order y post-order paso a paso
public class P04_Recorridos {

  static int paso;

  static void preOrden(Nodo<String> n, int nivel) {
    if (n == null) return;
    System.out.println("  ".repeat(nivel) + (++paso) + ") visita "
        + n.dato);  // 1° la raíz
    preOrden(n.izquierdo, nivel + 1);  // 2° izquierda
    preOrden(n.derecho, nivel + 1);  // 3° derecha
  }

  static void inOrden(Nodo<String> n, StringBuilder sb) {
    if (n == null) return;
    inOrden(n.izquierdo, sb);
    sb.append(n.dato).append(" ");
    inOrden(n.derecho, sb);
  }

  static void postOrden(Nodo<String> n, int nivel) {
    if (n == null) return;
    postOrden(n.izquierdo, nivel + 1);  // 1° izquierda
    postOrden(n.derecho, nivel + 1);  // 2° derecha
    System.out.println("  ".repeat(nivel) + (++paso) + ") visita "
        + n.dato); // 3° la raíz
  }

  public static void main(String[] args) {
    //            A
    //          /   \
    //         B     C
    //        / \   / \
    //       D   E F   G
    Nodo<String> arbol = new Nodo<>("A",
        new Nodo<>("B", new Nodo<>("D"), new Nodo<>("E")),
        new Nodo<>("C", new Nodo<>("F"), new Nodo<>("G")));

    System.out.println("PRE-ORDER (raíz, izquierda, derecha):");
    paso = 0;
    preOrden(arbol, 0);

    StringBuilder sb = new StringBuilder();
    inOrden(arbol, sb);
    System.out.println("\nIN-ORDER (izquierda, raíz, derecha): " + sb);

    System.out.println("\nPOST-ORDER (izquierda, derecha, raíz):");
    paso = 0;
    postOrden(arbol, 0);
  }
}
```

▶ **Resultado esperado en la terminal**

```
PRE-ORDER (raíz, izquierda, derecha):
1) visita A
  2) visita B
    3) visita D
    4) visita E
  5) visita C
    6) visita F
    7) visita G

IN-ORDER (izquierda, raíz, derecha): D B E A F C G

POST-ORDER (izquierda, derecha, raíz):
    1) visita D
    2) visita E
  3) visita B
    4) visita F
    5) visita G
  6) visita C
7) visita A
```

Análisis: en **pre-order** la raíz A es la **primera** visitada; en **post-order** es la **última**, porque primero se terminan sus dos subárboles. Compara con la tabla de la sección 2.4.

Práctica 05. Aplicación: árbol de expresión (Semana 15)

Una expresión aritmética se puede guardar en un árbol: los **operadores** son nodos internos y los **números** son hojas. Cada recorrido produce una notación distinta y la **evaluación** es un recorrido **post-order**.

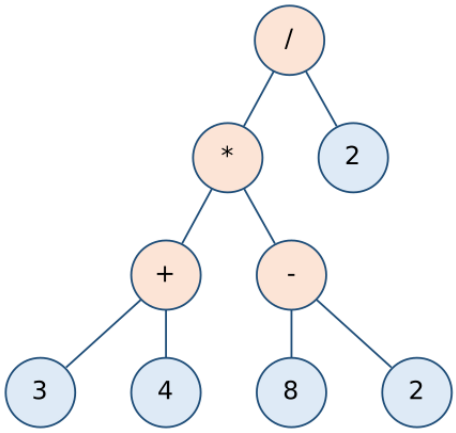


*Figura 4. Árbol de la expresión ((3 + 4) * (8 - 2)) / 2.*

Archivo: src/P05_ArbolExpresion.java

```
// P05_ArbolExpresion.java - Aplicación: árbol de una expresión aritmética
public class P05_ArbolExpresion {

  // Pre-order -> notación PREFIJA (polaca)
  static String prefija(Nodo<String> n) {
    if (n.esHoja()) return n.dato;
    return n.dato + " " + prefija(n.izquierdo) + " " + prefija(n.derecho);
  }

  // In-order -> notación INFIJA (la que usamos en matemáticas)
  static String infija(Nodo<String> n) {
    if (n.esHoja()) return n.dato;
    return "(" + infija(n.izquierdo) + " " + n.dato + " " + infija(n.derecho) + ")";
  }

  // Post-order -> notación POSTFIJA (la que usan las calculadoras y compiladores)
  static String postfija(Nodo<String> n) {
    if (n.esHoja()) return n.dato;
    return postfija(n.izquierdo) + " " + postfija(n.derecho) + " " + n.dato;
  }

  // EVALUAR también es post-order: primero se calculan los hijos y al final el operador
  static double evaluar(Nodo<String> n) {
    if (n.esHoja()) return Double.parseDouble(n.dato);
    double a = evaluar(n.izquierdo);
    double b = evaluar(n.derecho);
    double r = switch (n.dato) {
      case "+" -> a + b;
      case "-" -> a - b;
      case "*" -> a * b;
      case "/" -> a / b;
      default -> throw new IllegalArgumentException("Operador no válido: " + n.dato);
    };
    System.out.println("   calcula " + a + " " + n.dato + " " + b + " = " + r);
    return r;
  }

  public static void main(String[] args) {
    //              /
    //            /   \
    //           *     2
    //         /   \
    //        +     -
    //       / \   / \
    //      3   4 8   2
    Nodo<String> expr = new Nodo<>("/",
        new Nodo<>("*",
            new Nodo<>("+", new Nodo<>("3"), new Nodo<>("4")),
            new Nodo<>("-", new Nodo<>("8"), new Nodo<>("2"))),
        new Nodo<>("2"));

    System.out.println("Prefija  (pre-order) : " + prefija(expr));
    System.out.println("Infija   (in-order)  : " + infija(expr));
    System.out.println("Postfija (post-order): " + postfija(expr));
    System.out.println("\nEvaluación (post-order):");
    System.out.println("Resultado = " + evaluar(expr));
  }
}
```

▶ **Resultado esperado en la terminal**

```
Prefija  (pre-order) : / * + 3 4 - 8 2 2
Infija   (in-order)  : (((3 + 4) * (8 - 2)) / 2)
Postfija (post-order): 3 4 + 8 2 - * 2 /

Evaluación (post-order):
   calcula 3.0 + 4.0 = 7.0
   calcula 8.0 - 2.0 = 6.0
   calcula 7.0 * 6.0 = 42.0
   calcula 42.0 / 2.0 = 21.0
Resultado = 21.0
```

Análisis: para calcular `*` primero hay que conocer el valor de `+` y de `-`: por eso la evaluación es **post-order** (hijos primero, raíz al final). La notación postfija `3 4 + 8 2 - * 2 /` es la que usan las calculadoras científicas y los compiladores.

Práctica 06. Proyecto: Juego Sí/No que aprende (Semana 15)

Crearás el juego de la sección 2.5. El árbol se **carga** y se **guarda** en el archivo `datos/animales.txt` usando **pre-order**, y se cuentan los animales con **post-order**. Así, lo que el juego aprende no se pierde al cerrarlo.

Paso 1. Crear el archivo con el árbol inicial

Archivo: datos/animales.txt

```
P:¿Vive en el agua?
R:delfín
P:¿Tiene plumas?
R:loro
R:perro
```

Cada línea es un nodo escrito en **pre-order**: `P:` indica una pregunta (nodo interno con dos hijos: primero el «sí» y luego el «no») y `R:` una respuesta (hoja).

Paso 2. Crear el juego

Archivo: src/P06_JuegoSiNo.java

```
// P06_JuegoSiNo.java - PROYECTO: Juego Sí/No "Adivina el animal" que APRENDE
// Cada nodo interno es una PREGUNTA; cada hoja es una RESPUESTA (un animal).
// Respuesta "s" -> rama izquierda   |   respuesta "n" -> rama derecha
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Iterator;
import java.util.List;
import java.util.Scanner;

public class P06_JuegoSiNo {

  static final String ARCHIVO = "datos/animales.txt";
  static Scanner sc = new Scanner(System.in);

  // ---------- CARGAR el árbol desde el archivo (lectura en PRE-ORDER) ----------
  static Nodo<String> cargar(Iterator<String> lineas) {
    String linea = lineas.next();
    Nodo<String> n = new Nodo<>(linea.substring(2));
    if (linea.startsWith("P:")) {  // una pregunta siempre tiene 2 hijos
      n.izquierdo = cargar(lineas);
      n.derecho = cargar(lineas);
    }
    return n;
  }

  // ---------- GUARDAR el árbol en el archivo (escritura en PRE-ORDER) ----------
  static void guardar(Nodo<String> n, PrintWriter out) {
    if (n.esHoja()) {
      out.println("R:" + n.dato);
    } else {
      out.println("P:" + n.dato);
      guardar(n.izquierdo, out);
      guardar(n.derecho, out);
    }
  }

  // ---------- JUGAR: se baja por el árbol de forma recursiva ----------
  static void jugar(Nodo<String> n) {
    if (!n.esHoja()) {                                  // es una pregunta
      if (preguntar(n.dato)) jugar(n.izquierdo);
      else jugar(n.derecho);
      return;
    }
    if (preguntar("¿Es un(a) " + n.dato + "?")) {       // es una respuesta
      System.out.println("¡Adiviné!");
    } else {
      aprender(n);
    }
  }

  // ---------- APRENDER: la hoja se convierte en una nueva pregunta ----------
  static void aprender(Nodo<String> hoja) {
    System.out.print("Me rindo. ¿En qué animal pensabas? ");
    String nuevo = sc.nextLine().trim();
    System.out.print("Escribe una pregunta que diferencie a " + nuevo + " de "
        + hoja.dato + ": ");
    String pregunta = sc.nextLine().trim();
    boolean siParaNuevo = preguntar("Para " + nuevo + ", ¿la respuesta es sí?");
    Nodo<String> viejo = new Nodo<>(hoja.dato);
    Nodo<String> otro = new Nodo<>(nuevo);
    hoja.dato = pregunta;
    hoja.izquierdo = siParaNuevo ? otro : viejo;
    hoja.derecho = siParaNuevo ? viejo : otro;
    System.out.println("¡Gracias! Ya aprendí a reconocer: " + nuevo);
  }
```

```
  static boolean preguntar(String texto) {
    System.out.print(texto + " (s/n): ");
    String r = sc.nextLine().trim().toLowerCase();
    return r.startsWith("s");
  }

  // POST-ORDER: cuenta los animales (hojas) que el juego conoce
  static int contarAnimales(Nodo<String> n) {
    if (n.esHoja()) return 1;
    return contarAnimales(n.izquierdo) + contarAnimales(n.derecho);
  }

  public static void main(String[] args) throws IOException {
    List<String> lineas = Files.readAllLines(Path.of(ARCHIVO));
    Nodo<String> raiz = cargar(lineas.iterator());
    System.out.println("=== ADIVINA EL ANIMAL ===");
    System.out.println("Conozco " + contarAnimales(raiz)
        + " animales. Piensa en uno...");

    do {
      jugar(raiz);
    } while (preguntar("¿Jugamos otra vez?"));

    try (PrintWriter out = new PrintWriter(Files.newBufferedWriter(Path.of(ARCHIVO)))) {
      guardar(raiz, out);
    }
    System.out.println("Ahora conozco " + contarAnimales(raiz)
        + " animales. Árbol guardado en " + ARCHIVO);
  }
}
```

Paso 3. Jugar y enseñarle dos animales nuevos

En esta partida de ejemplo se pensó en un **pingüino** y luego en un **gato** (las respuestas escritas por el usuario aparecen después de cada pregunta):

▶ **Partida de ejemplo**

```
=== ADIVINA EL ANIMAL ===
Conozco 3 animales. Piensa en uno...
¿Vive en el agua? (s/n): n
¿Tiene plumas? (s/n): s
¿Es un(a) loro? (s/n): n
Me rindo. ¿En qué animal pensabas? pingüino
Escribe una pregunta que diferencie a pingüino de loro: ¿Puede volar?
Para pingüino, ¿la respuesta es sí? (s/n): n
¡Gracias! Ya aprendí a reconocer: pingüino
¿Jugamos otra vez? (s/n): s
¿Vive en el agua? (s/n): n
¿Tiene plumas? (s/n): n
¿Es un(a) perro? (s/n): n
Me rindo. ¿En qué animal pensabas? gato
Escribe una pregunta que diferencie a gato de perro: ¿Ladra?
Para gato, ¿la respuesta es sí? (s/n): n
¡Gracias! Ya aprendí a reconocer: gato
¿Jugamos otra vez? (s/n): n
Ahora conozco 5 animales. Árbol guardado en datos/animales.txt
```

Al terminar, `datos/animales.txt` contiene el árbol aprendido:

datos/animales.txt después de jugar

```
P:¿Vive en el agua?
R:delfín
P:¿Tiene plumas?
P:¿Puede volar?
R:loro
R:pingüino
P:¿Ladra?
R:perro
R:gato
```

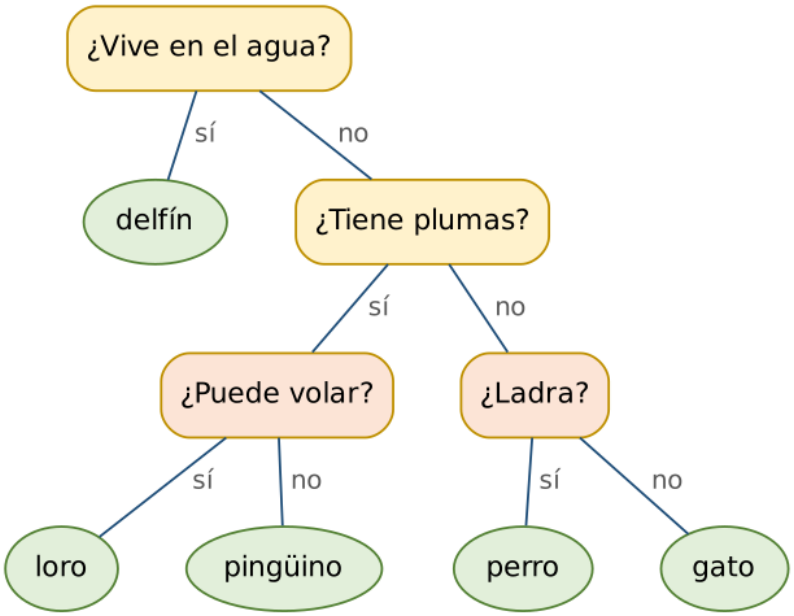


Figura 5. Árbol después de aprender «pingüino» y «gato» (nuevas preguntas en naranja).

Análisis: cada vez que el juego falla, una **hoja** se transforma en una **pregunta** con dos hijos: el árbol crece y el juego se vuelve más «inteligente». Los métodos `cargar`, `guardar`, `jugar` y `contarAnimales` son todos recursivos.

> 💡 **Consejo:** Para volver a empezar desde cero, reemplaza el contenido de `datos/animales.txt` por las 5 líneas del Paso 1.

> ⚠ **Atención:** Si al escribir «pingüino» la ü o la ñ se guardan mal, ejecuta `chcp 65001` en la terminal antes de iniciar el juego.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
`StackOverflowError`	Falta el caso base o la llamada no se acerca a él.	Revisa que exista un `if` que termine sin llamarse y que el parámetro cambie (`n - 1`).
`NullPointerException` al recorrer el árbol	Se usó `n.dato` cuando `n` era `null`.	Empieza cada método recursivo con `if (n == null) return …;`.
El árbol solo muestra la raíz	Se olvidó asignar el resultado: `insertar(n.izquierdo, v);`	Usa `n.izquierdo = insertar(n.izquierdo, v);`.
`cannot find symbol: class Nodo`	`Nodo.java` no está en `src` o tiene otro nombre.	Crea `src/Nodo.java` con la clase `Nodo`.
`NoSuchFileException: datos\animales.txt`	Se abrió otra carpeta en VS Code.	Abre la carpeta del proyecto y verifica que exista `datos/animales.txt`.
`NoSuchElementException` al cargar el juego	El archivo `animales.txt` está incompleto (una pregunta sin sus dos hijos).	Restaura las 5 líneas originales del archivo.

Parte 5. Ejercicios propuestos

1. Escribe dos métodos recursivos: `contar(String texto, char c)`, que cuente cuántas veces aparece un carácter, e `invertir(String texto)`, que devuelva el texto al revés. Prueba con "inteligencia artificial".
2. Resuelve las **Torres de Hanói** de forma recursiva para 3 discos, mostrando cada movimiento («Mover disco 1 de A a C»). Verifica que el total de movimientos sea $2^n - 1$.
3. Agrega a `ArbolBinario` los métodos recursivos `suma()`, `minimo()`, `maximo()` y `nodosEnNivel(int k)`. Pruébalos con los valores de la Práctica 03.
4. Construye el árbol de la expresión `(5 - 1) * (2 + 3) + 4`. Muestra su notación prefija y postfija, y evalúalo con el método de la Práctica 05.
5. Modifica el Juego Sí/No para que: a) muestre la lista de todos los animales que conoce (recorrido de las hojas), b) muestre la altura del árbol y c) cambie de tema a «Adivina la carrera de SENATI» con un archivo `carreras.txt`.

Preguntas de reflexión

1. ¿Qué pasaría con factorial si el caso base fuera `if (n == 0)` y lo llamaras con un número negativo?
2. ¿Por qué los árboles son estructuras «naturalmente recursivas»?
3. ¿Por qué se usa pre-order para guardar el árbol del juego y no in-order?
4. ¿En qué se parece el juego Sí/No a un sistema que diagnostica fallas de una computadora?

Lista de cotejo

- ☐ Identifico el caso base y el caso recursivo de una función (Práctica 01).
- ☐ Comparo soluciones recursivas e iterativas y reconozco el StackOverflowError (Práctica 02).
- ☐ Explico raíz, nodo, hoja, nivel y altura de un árbol.
- ☐ Creé las clases `Nodo` y `ArbolBinario` con métodos recursivos (Práctica 03).
- ☐ Ejecuté y explico los recorridos pre-order, in-order y post-order (Práctica 04).
- ☐ Evalué un árbol de expresión con post-order (Práctica 05).
- ☐ Jugué y le enseñé animales nuevos al juego Sí/No; el árbol se guardó en el archivo (Práctica 06).
- ☐ Resolví al menos tres ejercicios propuestos.